

K-Nearest Neighbors

Sean Hellingman ©

Multivariate Statistics for Applied Data Science (ADSC 4050)

shellingman@tru.ca

Winter 2026



THOMPSON RIVERS UNIVERSITY

Topics

- 2 Recap
- 3 Introduction
- 4 Measuring Similarity
- 5 kNN Classification
- 6 Choosing k
- 7 kNN Regression
- 8 Applications
- 9 Conclusions

Recap

- Last time we covered **PCA II**:
 - Implementation in R.
 - Number of PCs.
 - Applications.

Introduction

- We have covered some complex and fairly simple approaches to regression and classification problems.
- We can use distance measures between *similar* observations to estimate classes or response variables.
- k -nearest neighbors (kNN) may be implemented to achieve this.

K-Nearest Neighbors

- **k-nearest neighbors (kNN)** is a very simple algorithm in which each observation is predicted based on its *similarity* to other observations.
- This approach **cannot** be saved or represented as a closed-form model.
 - It is a *lazy learner* and needs to be executed every time that it is needed.
- May be used for classification and regression problems.
 - Also for missing observations.

Measuring Similarity

- kNN identifies the k observations that are most *similar* to the object being predicted.
- Distance or dissimilarity measures are used to obtain pairwise differences between objects.
- Distance & dissimilarity measures:
 - Euclidean distance.
 - Manhattan distance.
 - Jaccard dissimilarity.
 - Minkowski distance.

Recall: Euclidean Distance

- Euclidean Distance generalizes to many dimensions and can be useful for numeric, binary, and ordinal data.
 - Values may be positive or negative.
 - In R `dist()`. In Python `scipy.spatial.distance_matrix`
- Euclidean Distance (ED) between objects i and h across p dimensions is:

$$ED = \sqrt{\sum_{j=1}^p (a_{hj} - a_{ij})^2} \quad (1)$$

- **The distance depends on the scale of the variables.**
- Generally inappropriate for community data.

Recall: Manhattan Distance

- Manhattan Distance simply sums the absolute differences between values.
 - Values may be positive or negative.
 - In R `dist(A,method="manhattan")`.
- Manhattan Distance (MD) between objects i and h across p dimensions is:

$$MD = \sum_{j=1}^p |(a_{hj} - a_{ij})| \quad (2)$$

- **The distance depends on the scale of the variables.**
- Generally inappropriate for community data.

Recall: Jaccard Similarity & Dissimilarity I

- The **Jaccard Similarity** measures the *overlap* in attributes between two objects.
- Contingency table:

		Object B	
		Present	Absent
Object A	Present	a	b
	Absent	c	d

- a is the number of common attributes (overlap).
- b number of attributes in A but not in B.
- c number of attributes in B but not in A.
- d attributes that are missing from both (if applicable)

Recall: Jaccard Similarity & Dissimilarity II

- The Jaccard similarity (S_J) is calculated as follows:

$$S_J = \frac{a}{a + b + c} \quad (3)$$

- This is a metric measure that is bounded by 0 and 1.
- Since the Jaccard similarity is bounded by 1, it is easily converted to a dissimilarity measure by subtraction.
- The Jaccard dissimilarity (D_J) is calculated as follows:

$$D_J = 1 - S_J = \frac{b + c}{a + b + c} \quad (4)$$

- This measures the proportion of attributes that do not overlap in the objects.

Recall: Minkowski Distance

- Minkowski Distance a generalization of the Euclidean and Manhattan distance measures.
 - Adds an additional parameter p that specifies the root and power to be applied.
 - In R `dist(A,method="minkowski",p=2)`.
- Minkowski Distance ($D_{Minkowski}$) between objects i and h across m dimensions is:

$$D_{Minkowski} = \left(\sum_{j=1}^m |(a_{hj} - a_{ij})|^p \right)^{\frac{1}{p}} \quad (5)$$

- **The distance depends on the scale of the variables.**
- $p = 1$: Manhattan distance.
- $p = 2$: Euclidean distance.
- If $p < 1$ then the measure is not a metric.

***k*-Nearest Neighbors**

***k*-Nearest Neighbors Classification**

- In R we can use the `knn()` function from the *class* package.
 - In Python.

- Implementation:

```
knn(  
  train = scaled.train,  
  test = scaled.test,  
  cl = train.vector.classes,  
  k = k)
```

- Returns a vector of predicted classes.
- Note: *This is based off of a Euclidean distance.*

Illustrative Example 1

- Run the provided code to visualize two-dimensional kNN classification.
- What do you notice?

Example 1

- Import the *Penguins3.csv* file to conduct the following analysis:
 - 1 Scale your data and perform a 75/25 train/test split.
 - 2 Perform kNN with $k = 3$
 - 3 Perform kNN with $k = 11$
 - 4 Perform kNN with $k = 41$
 - 5 Compare the accuracies
- What did you notice?

Choosing k

Choosing k

- **The performance of this approach is very sensitive to the choice of k .**
- Low values typically overfit and high values underfit.
- We can use a grid search to help to select k .
- *Typically, we only consider odd values of k to avoid ties.*

Illustrative Example 2

- Run the provided code to determine the best value of k for the training set.
- Does this value generalize well to the unseen data?

Example 2

- Use the scaled *Penguins3.csv* dataset to conduct the following analysis:
 - 1 Perform a grid search to determine the best k based on cross validation of the training set.
 - 2 Plot your process.
 - 3 Does this k generalize well to the testing set?

***k*-Nearest Neighbors Regression**

- The algorithm may also be used on continuous response variables.
- Again, the results are sensitive to the choice of k .
- Predicts the value based on the average of the k nearest objects.

Illustrative Example 3

- Run the provided code to determine the optimal value of k to predict the Petal.Length of irises.
- How well does this generalize?

Example 3

- Import the *Wages.csv* dataset and run the provided code to scale and split the dataset for the following analysis:
 - 1 Conduct a grid search for the best value of k based on cross-validation.
 - 2 Plot your results.
 - 3 How well does this k generalize to unseen data?

Other Distance Measures

- All of the examples we have covered use the Euclidian distance.
- You can use the `kkn` package to explore other distance measures.
- **Run the code in Illustrative Example 4 to see how this works.**

Applications

Applications of kNN

- kNN may be used as a regression or classification tool.
 - It is a *lazy learner* so you will need to run the entire process every time that it is used.
- This algorithm does work well in certain conditions.
 - Small to medium sized datasets with a clear response variable.
 - Not overly complex relationship structures among variables.
- **May also be very useful for dealing with missing observations.**

Missing Values

- Throughout your studies, you have probably encountered the following ways of dealing with missing observations:
 - 1 Remove missing observations.
 - 2 Replace missing observations with the column mean.
 - 3 Replace missing observations with regression imputation.
- Another approach is to use kNN to replace missing observations.

Example 4

- Import the *air.csv* dataset and do the following:
 - ① The *Ozone* variable is missing some observations, split your dataset into complete cases and rows with missing data.
 - ② Perform a grid search to determine the ideal value of k for a kNN on your complete cases.
 - ③ Apply that k to estimate the missing observations.
- Do these results seem reasonable?

Final Thoughts

- k -Nearest Neighbours is a *lazy learner* meaning to integrate this method into a pipeline, it will need to run every time.
- kNN has been shown to work well and is relatively easy to implement.
- Especially useful for imputing missing observations.

Next Lecture

- We will discuss **Boosting**.
- *Sequential ensembles*.
- Hyperparameters and tuning.

Exercise 1

- Import the ptitanic dataset from the *rpart.plot* package.
- The age variable is missing some observations.
- Can you use kNN to impute these missing observations?
- Note: You will need to pre-process the categorical variables (ordinal factors and one-hot encoding).
- What is the best value for k ?
- Do these imputations make sense?

References & Resources

- 1 Bakker, J. D. (2026) *Applied Multivariate Statistics in R*
 - 2 Peng R. D. (2020) *Exploratory Data Analysis with R*
 - 3 Timbers T., Campbell T., and Lee M. (2024) *Data science: A first introduction*. Chapman and Hall/CRC
 - 4 Bradley Boehmke B., and Greenwell B. (2020) *Hands-On Machine Learning with R*. Taylor & Francis
- caret
 - knn()
 - kkn